

Hardening API Authentication with Encrypted, Short-Lived Access Tokens Using JSON Web Encryption

Mrs.M.Shyni
Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varsha B
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—Token authentication is used in API which helps to identify the user who logs into the system. The usage of conventional JWT means that it is not encrypted, this means that anyone who intercepts the token will be able to decode the information contained in it such as user id, token id, expiry time, privileges etc. The purpose of this research is to talk about improving the token generation mechanism in FastAPI web application by means of usage of JWE technique that uses AES 256 GCM encryption and establishes fixed time period of 20 minutes. According to the analytical model, the precise token length is given (predicted length is 241 characters against measured length). Furthermore, the maximum of forgery probability is $q \cdot 2^{-128}$ and the average duration for which the token can be intercepted is $T/2 = 10$ minutes. By conducting experiments, the attacker managed to extract data from 4 out of 4 letters in the signed JWT but did not manage to decode letters in the form of JWE. Among 2,000 tokens, related to the cases of token usage with one flipped bit [2,000 tokens with a wrong decryption key and 2,000 tokens that became invalid], only valid tokens were used. Moreover, 20,000 encryptions of the same texts resulted in obtaining 20,000 various IVs. The obtained data presented above can be compared with the previous results related to the signed JWT, which indicated a difference between the length of the text presented as a signed JWT and the text, issued in the form of the JWE.

Index Terms—API security, JSON Web Encryption, JWE, token authentication, credential confidentiality, AES-256-GCM, traffic analysis

I. INTRODUCTION

Almost all the APIs in existence nowadays provide a token after successful authentication allowing identification of the user in subsequent requests. The most common example is the JSON Web Token (JWT) which consists of three parts - header, payload, and signature. The signature guarantees that the payload has not been modified while payload remains only base64url encoded. Although implicating the token via interception or through misconfiguration or hacking won't generate another token, as anyone suffering from a mishap will be able to access the token. The claims in the token such as the user ID, token ID, validity time, and privilege flag could therefore be read by anyone who intercepts the token..

This research is part of a more extensive research project that investigates lightweight API security and includes an implementation of a real-time detection system that focuses on five categories from the OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object level authorization, and broken function level authorization (2). The detection system takes place on a simulated FastAPI backend, recognizes each incoming request in real-time through fast checks, while leaving analysis based on machine learning for after-the-fact analysis of suspicious requests. This paper is a continuation of this project, which is aimed at hardening the credential layer that the traffic relies on: access tokens generated following a successful authentication. The detection system in (2) recognizes suspicious traffic flows but does not take into account the security of tokens used in this traffic.

This issue of integrity separation, which is provided by the received JWT, is addressed by the established IETF specification called JSON Web Encryption (JWE) [1]. The main contributions of this paper are as follows:

- Development of a threat model and a development of the analytical model of the mechanism of JWE token giving the closed formulas for token size, forgery probability, key search effort, probability of IV collision and interception window for the token which has been intercepted.
- Creation of a reference implementation of the short-term encrypted token in a FastAPI backend which can be easily used in place of the signed JWT issuing without any changes needed from the client side.
- Development of a reproducible benchmark which measures in size, latency, confidentiality, tampering refusal, wrong key refusal, expiration honoring and IV unique, which can be compared with the analytical model.

- Discovery of the fact that ciphertext size allows revealing the privilege flag in correlation with the constant padding length and its overhead.

The rest of the paper is structured as follows. Related work is discussed in Section II. The threat model is presented in Section III. The mechanism is described in Section IV. The analytical model is developed in Section V. The experimental setup is described in Section VI. The results are presented in Section VII. Discussion and comparison to conventional JWT issuance is provided in Section VIII. Limitations are discussed in Section IX, and conclusions are given in Section X.

II. RELATED WORK

JSON Web Token (JWT) [3] describes a compact and URL-safe encoding for claims carried between two entities, usually digitally signed with either HMAC or RSA to ensure integrity. A digitally signed-only JWT (JWS) [7] lacks confidentiality: its payload is only base64url encoded, easily decodable without the signing key. Current best practices for using JSON Web Tokens [19] and for bearer tokens [21] in general focus on selecting an algorithm and verifying integrity rather than protecting the content of the token.

JSON Web Encryption (JWE) [1] guarantees confidentiality through a compact format who contains five different parts: protected header, encrypted key, IV, ciphertext, and authentication tag. The algorithms use by JWE are defined in JSON Web Algorithms (JWA) [8], and the keys are provided in JSON Web Keys (JWK) [9]. The scheme used in this case is AES-256-GCM, which is an authenticated encryption scheme that guarantees both confidentiality and integrity of the data by means of a single cryptographic primitive [4], [18]; AES is standardized in FIPS PUB 197 [16] and its key management is described in NIST SP 800-57 [17].

One of the categories API vulnerability is Broken Authentication, and the examples include credential stuffing, lack of proper token generation, and the case when the contents of the token are exposed to malicious third parties. Most of the important recommendation deals with detection of stolen tokens abuse, while there is not much information available about what the tokens are capable of revealing to the attacker until the abuse has already taken place.

A wider array of standards associated with OAuth 2.0 provides additional context. The framework covering OAuth 2.0 [6], its threat model [10], and the security protocol analysis [11] describe the types of vulnerabilities in authorization flows apart from keeping the confidentiality of the token. PKCE [12] and Token Introspection [13] address the problems that are related but different than those mentioned above while OpenID Connect [14] provides identity claims. Protection against token interception at the transport layer is achieved by using TLS 1.3 [15] and this protection complements the protection of the payload since the token is store in a decrypted form in proxy servers, logs, as well as browser storage. Short-lived credentials are the well-known design principle that limits the lifetime of stolen credentials [6]. The passwords that are hashed using bcrypt [20] are purposely expensive, which cost is referred to in Section VII when determining token overhead. Unlike the usual token recommendation, this work compares the present solution against traffic-level detector [2] at the same API testbed.

III. THREAT MODEL AND SECURITY GOALS

The asset consists of the four claims which the token carries: sub, jti, exp, and is_admin. Let F be the set containing these assets and $|F| = 4$. The server stores only one 256-bit symmetric key which never leaves the server. There are three security objectives formulated: G1, ensuring the secrecy of F from an adversary which does not have access to the key; G2, providing the integrity of the token so that any modification of the token is detected; and G3, a limited useful lifetime for a compromised token. The adversaries considered are summarized in Table I.

TABLE I
THREAT MODEL

Adversary	Capability	Goal
A1 Passive interceptor	Observes tokens in transit, in logs, or at rest; no key	G1
A2 Active tamperer	Modifies any part of a token and replays it	G2
A3 Key guesser	Tries to decrypt or forge tokens by guessing the key	G1, G2
A4 Late replayer	Replays a captured, unmodified token after a delay	G3

A token replayed unmodified within its lifetime is accepted, because a bearer token is not bound to a client. The mechanism bounds this risk through the short expiry (G3) but does not eliminate it.

IV. PROPOSED MECHANISM

A. Token Issuance

After successful authentication, the server will create the claim object with 4 attributes. The given claim object is then converted into JSON format and encrypted using the JWE using the dir key wrapping mechanism (in this mechanism, the symmetric key is used directly without any wrapping of keys) and the encryption algorithm used is A256GCM. The issuance is given in Algorithm 1.

Algorithm 1: Token issuance

Input: user id u , privilege flag a , server key K , lifetime T .

1. $jti \leftarrow$ new random UUID.
2. $exp \leftarrow$ current time + T .
3. $claims \leftarrow \{sub: u, jti: jti, exp: exp, is_admin: a\}$.
4. $IV \leftarrow$ fresh random value.
5. $(C, tag) \leftarrow \text{AES-256-GCM-Encrypt}(K, IV, \text{JSON}(claims))$.
6. Return $header.empty().IV.C.tag, \text{base64url-encoded}$.

B. Token Verification

The token is decrypted at the server using the same key for every request. In the case of unsuccessful decryption or signature verification, the request would be denied. On the other hand, if decryption was done successfully but the token has expired, then the request still would be denied. Algorithm two provides a summary for verification.

Algorithm 2: Token verification

Input: token, server key K .

1. Split token into header, key, IV , C , tag .
2. Decrypt C under K , IV ; verify tag; else reject.
3. Parse JSON claims; on failure, reject.
4. If exp is not later than now, reject.
5. Return claims.

C. Short Expiry

Tokens have a lifespan of $T=20$ minutes. This restricts the period in which any intercepted tokens will remain valid, regardless of when the interception is noticed. The precise period is examined in Section V-D.

D. Fixed-Length Claim Padding (Extension)

The claim has its contents encoded, but its size is transparent. This is because the size of JSON text representing a privileged token is different from the size of the JSON text that represents a common token. To alleviate this issue, the serialized claims are padded with an additional field so that the plaintext is always of size P . The complete process is shown in Fig. 1.

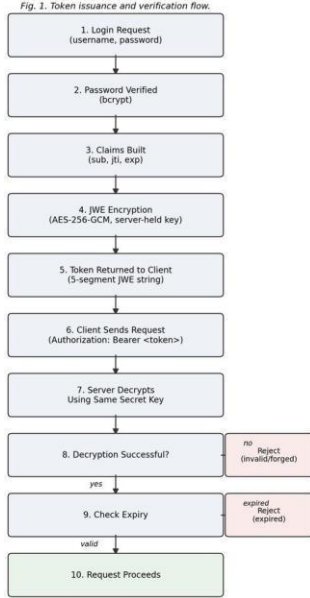


Fig. 1. Token issuance and verification flow.

V. ANALYTICAL MODEL

A. Token Size

The unpadded base64url representation of n bytes yields the character length given by

$$L(n) = \lceil 4n/3 \rceil \quad (1)$$

characters. A compact JSON Web Encryption (JWE) token consists of a header of h bytes, an encrypted key of k bytes (with k equal to 0 when the encryption is performed by means of a symmetric key), an initialization vector (IV) of v bytes, an encrypted message of m bytes (where $m = p$ since Galois/Counter Mode (GCM) retains the size of plaintext), and a tag of τ bytes, as represented in the following equation (note that the token parts are divided by four dots):

$$S_{JWE} = L(h) + L(k) + L(v) + L(m) + L(\tau) + 4 \quad (2)$$

In turn, a signed JSON Web Token (JWT) using the HMAC SHA256 algorithm (HS256) includes a header of h' bytes, a payload of P bytes, a 32-byte signature, and two dots

$$S_{JWS} = L(h') + L(p) + L(32) + 2 \quad (3)$$

Therefore, the relative size of overhead of encryption may be defined as

$$\delta S = (S_{JWE} - S_{JWS}) / S_{JWS} \quad (4)$$

B. Forgery and Key Search

For a t -bit authentication tag, the probability a single forged token is accepted is on the order of 2^{-t} [4], so an adversary making q forgery attempts succeeds with probability at most

$$P_{\text{forge}}(q) \leq q \cdot 2^{-t} \quad (5)$$

For $t = 128$ and $q = 240$ attempts, P_{forge} is at most 2^{-88} , about 3.2×10^{-27} . Exhaustive search of a k -bit key needs on average 2^{k-1} trials, so at R trials per second

$$T_{\text{search}} = 2^{k-1} / R \quad (6)$$

For $k = 256$ and $R = 1012$ trials/s, T_{search} is about 5.8×10^{64} seconds, or about 1.8×10^{57} years.

C. IV Collision

Each token uses a fresh random IV of b bits. By the birthday bound, the probability that any two of n tokens share an IV is approximately

$$p_{IV}(n) \approx n(n-1) / 2^{b+1} \quad (7)$$

and the number of tokens issuable under one key before this probability reaches a target p is

$$n^*(p) = \sqrt{2^{b+1} \cdot p} \quad (8)$$

With the measured $b = 128$, $p_{IV}(20,000)$ is about 5.9×10^{-31} , and $n^*(2^{-32})$ is about 4.0×10^{14} tokens, so key lifetime is not limited by IV collisions at this system's scale.

D. Exposure Window

Suppose a token of lifetime T is intercepted at a time u distributed uniformly over its lifetime. The remaining useful time is $W = T - u$, so

$$E[W] = T/2 \quad (9a)$$

$$P(\text{valid after delay } d) = \max(0, 1 - d/T) \quad (9b)$$

For $T = 20$ minutes, $E[W] = 10$ minutes and the worst case is 20 minutes. Relative to a reference lifetime T_{ref} , the exposure window is reduced by the factor T_{ref}/T ; for a 24-hour token this factor is 72.

E. Confidentiality Metrics

Let $1[f]$ equal 1 if field f in F can be recovered without the key. The exposure ratio is

$$E_x = (1/|F|) \sum_{f \in F} 1[f] \quad (10)$$

For a length-based classifier predicting privilege class C from token length Λ , with $N(l, c)$ tokens of length l and class c among N tokens, accuracy and leaked information are

$$A = (1/N) \sum_l \max_c N(l, c) \quad (11a)$$

$$I(\Lambda; C) = H(C) - H(C|\Lambda) \quad (11b)$$

For balanced classes, $H(C) = 1$ bit and chance accuracy is 0.5. Padding every plaintext to a fixed length P gives a padding field of length

$$\pi = P - m_0 \geq 0 \quad (12)$$

where m_0 is the unpadded plaintext length, making Λ constant, so $I(\Lambda; C) = 0$.

VI. EXPERIMENTAL SETUP

Every experiment utilizes the same token module call, the same key, and a backend lifetime of 20 minutes, and a single script is executed to recreate the numbers with the same random seed of 42. The baseline uses the HS256 JWT signature with four identical claims. All measurements are based on $n = 10,000$ measurements being made while 300 warmed-up executions are used; results include mean, median, standard deviation, and 95th and 99th percentiles. The experiments were conducted in a single-core environment using Python 3.12 and python-jose 3.3.0. The mean bcrypt verification time from five runs is utilized as the reference cost for login. For the rejection experiment, $N = 2,000$ tokens were used for the tampering experiment in which one random bit was manipulated in every header/IV/ciphertext/tag (500 instances). The wrong-key decryption was conducted using a random key of 32 characters and an expiry experiment was done with tokens with an expiry of between 1 second and 24 hours (rejection) as well as within the lifetime (for acceptance) along with one live 2 seconds lifetime token. An IV length experiment used each claim and was encrypted 20,000 times, and a length leak experiment was conducted with 1,000 normal tokens and 1,000 administrator tokens.

VII. RESULTS

A. Structure and Size

Every token contained five segments that can be separated by a dot while signed JWTs had three segments. In the section about key encryption, there was nothing since it was the case of dir. Table II provides information on the real size compared to the theoretical values of the size described by Eqs. (2) and (3). The real and estimated size precisely match (JWE equals 241, JWT equals 201) leading to the token size of 40 characters (Eq. 4).

TABLE II
TOKEN SIZE: PREDICTED VS. MEASURED

Component	Bytes	Chars
JWE header	29	39
JWE encrypted key	0	0
JWE IV	16	22
JWE ciphertext	115	154
JWE tag	16	22
JWE total (pred./meas.)	—	241 / 241
JWS header	27	36
JWS payload	90	120
JWS signature	32	43
JWS total (pred./meas.)	—	201 / 201

B. Latency

The timings can be observed from Table III, while Figure 2(b) depicts the timings through graphical representation. The generating of JWE consumed 54.14 μ s versus 23.42 μ s of time for the signed JWT, thus giving a lag of around 31 μ s. The verification of JWE consumed 35.82 μ s as opposed to 39.86 μ s for signed JWT, meaning that it does not take longer; however, it should not be treated as the benefit of encryption since different groups of libraries were used along with claim validation for JWT. The single verification takes 36 μ s, which means that measurements amount to around 27,900 verifications per second per core.

TABLE III
LATENCY (MS, $N = 10,000$)

Operation	Mean	Median	P95	P99
JWS issue	23.42	20.90	34.82	53.53
JWE issue	54.14	47.92	81.40	113.11
JWS verify	39.86	35.86	59.58	81.53
JWE verify	35.82	31.98	54.77	81.52
bcrypt (ms)	297.33	—	—	—

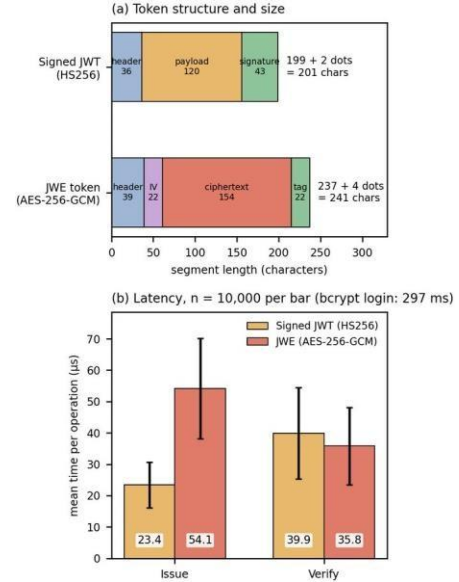


Fig. 2. (a) Token structure and size; (b) issuance/verification latency.

C. Security Experiments

The results of the security tests are depicted in Table IV. The interception of four out of four claims made by each signed JWTs took place ($Ex = 1.00$), but there were no claims recovered from JWE tokens ($Ex = 0.00$) in the experiments carried out. All malformed, invalid, and expired tokens were rejected. Notice that all valid tokens were accepted. Out of 20,000 of the identical claims being encrypted using the same secret key, each encryption produced a different pair of IV and ciphertext, thus confirming that $p_{IV} \approx 5.9 \times 10^{-31}$, as shown in Equation (7).

TABLE IV
SECURITY EXPERIMENTS

Experiment	N	Result
Claim fields recoverable, JWT	1,000	4 of 4 ($Ex=1.00$)
Claim fields recoverable, JWE	1,000	0 of 4 ($Ex=0.00$)
Bit flip (header/IV/CT/tag)	2,000	2,000 rejected
Bit flip, JWT payload	2,000	2,000 rejected
Wrong-key decryption	2,000	2,000 rejected
Expired tokens	2,000	2,000 rejected
Valid tokens	2,000	2,000 accepted
Live 2-s expiry test	1	ok@0s, rej.@3s
Distinct IVs / ciphertexts	20,000	20,000 / 20,000

D. Length Leakage

The JWE payload has been encrypted, which makes it unreadable. However, this process does not hide the length of tokens. The average token length is measured at 241 bytes, while the admin token length is 239 bytes because of the noticeable difference in length between the string values true and false. A simple classification method based on token length was able to classify the privilege type of tokens correctly at a 100% rate (Eq. 11a). Thus, the rate of information leakage is 1

bit per token.

Once the tokens were padded to a constant length $P=192$ bytes, their length grew to 343 bytes, which caused the accuracy of the classifier to decrease to the level of 50%, which is equivalent to blind guessing. In this case, the information leakage would equal 0 bits.

TABLE V
LENGTH LEAKAGE OF THE PRIVILEGE FLAG

Variant	Chars	Lengths	Accuracy
Unpadded	241 / 239	2	100%
Padded ($P=192$)	343	1	50%

VIII. DISCUSSION AND COMPARISON

The results confirm the goals of confidentiality and integrity, G1 and G2: For instance, the token does not display any of its four claim fields for parties not possessing the key, while all alterations are blocked in all experiments. The overhead is low in absolute terms, as the token takes up 19.9% more space, requires 31 μ s for issuance, does nearly the same for verification, which pales in comparison with the 297 ms necessary to hash a password during logging in. The length experiment proves that the confidentiality of content differs from the confidentiality of metadata: An attacker, while unable to obtain the value of the privilege field, may learn it by its length. Length padding removes this vulnerability with a minor cost. The analysis of the exposure window resolves the task described in goal G3: If T is set to be 20 minutes, the intercepted token can be expected to have a remaining useful life of 10 minutes, which is 72 times smaller than for a 24-hour token.

TABLE VI
COMPARISON WITH CONVENTIONAL SIGNED JWT

Aspect	JWS	JWE (proposed)
Segments	3	5
Fields readable w/o key	4 of 4	0 of 4
Tamper detection	Yes (sig.)	Yes (auth. tag)
Privilege inferable	Yes	If unpadded
Size (chars)	201	241 (343 padded)
Issue time (μ s)	23.42	54.14
Verify time (μ s)	39.86	35.82
Client integration	—	None (drop-in)
Exposure window	Depends	10 min ($T=20$ min)

IX. LIMITATIONS

There are many constraints that need to be considered. First of all, a single symmetric key is used that does not change, and this means that if the key is revealed, then all tokens signed by the key will be revealed as well. Secondly, there is no refresh-token implementation, and therefore the users have to authenticate again every 20 minutes. Thirdly, even though the library creates IVs that are 128 bits long, it should have created 96-bit long IVs according to standard [8] for AES-GCM. However, in this case, it is acceptable since only one server signs and validates tokens. Fourthly, the benchmark was conducted on the padded version of the algorithm, and this version was not used on the endpoint yet. Moreover, every bearer token can be used more than one time within its validity period since the tokens do not correspond to any specific client. Finally,

the tampering test only alters one random bit to make it hardly resemble the real-life attack by an ordinary script.

timing depends on hardware and is done locally, all experiments were in-process too.

X. CONCLUSION AND FUTURE WORK

In this paper it has been proposed to use encrypted and temporary API access tokens created with JWE and AES-256-GCM whose life span is 20 minutes. An analytical model was created to determine the optimal size for the token (241 characters). During the research, it was noted that 0 of 4 claims are revealed to the interceptor in case of using the token while it is equal to 4 of 4 in case of using a signed JWT. More than 2000 counterfeit tokens were tested and rejected as well as 2000 tokens created using wrong key. Also, the number of rejected tokens created by using the expired tokens amounted to 2000. Finding the length of ciphertext made it known that figuring out the privilege bit can be achieved successfully every time. Since utilization of fixed-length padding made this impossible adding 102 characters makes this issue unthinkable. This paper synthesized while doing research on lightweight API security along with the companion detection system. Subsequent work will include adding padding to the deployed endpoint, key rotation, refresh tokens, 96-bit IVs to ensure interoperability, client binding in order to prevent replay attacks, and storing token usage data in the companion detection system.

ACKNOWLEDGMENT

Fig. 1 and Fig. 2 in this paper were generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

REFERENCES

- [1] M. Jones and J. Hildebrand, "JSON Web Encryption (JWE)," Internet Eng. Task Force, RFC 7516, 2015.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, 2015.
- [3] D. McGrew and J. Viegas, "The Galois/counter mode of operation (GCM)," in *Submission to NIST Modes of Operation Process*, vol. 20, 2004, pp. 0278–0070.
- [4] S. J. García Zambrano, D. A. Terán Piedra, E. F. Paredes Nacipucha, and D. A. Dávila Cárdenas, "Diseño de una metodología de Pentesting para APIs RESTful basada en los cinco principales riesgos del OWASP API Security Top 10 (2023)," Doctoral dissertation, UIDE, Quito, Ecuador, 2026.
- [5] D. Hardt, "The OAuth 2.0 authorization framework," Internet Eng. Task Force, RFC 6749, 2012.
- [6] M. Jones, "JSON Web Signature (JWS) unencoded payload option," Internet Eng. Task Force, RFC 7797, 2016.
- [7] M. Jones, "JSON Web Algorithms (JWA)," Internet Eng. Task Force, RFC 7518, May 2015.
- [8] M. Hofmeier and W. Hommel, "Enabling the JSON web signature format to support complex and identity-oriented non-web processes," in *Proc. Int. Conf. Innov. Community Services*, Cham, Switzerland: Springer Nature Switzerland, Sep. 2023, pp. 29–47.
- [9] T. Lodderstedt, M. McGloin, and P. Hunt, "OAuth 2.0 threat model and security considerations," Internet Eng. Task Force, RFC 6819, 2013.
- [10] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1204–1215.
- [11] N. Sakimura, J. Bradley, and N. Agarwal, "Proof key for

- code exchange by OAuth public clients,” Internet Eng. Task Force, RFC 7636, 2015.
- [12] J. Richer, “OAuth 2.0 token introspection,” Internet Eng. Task Force, RFC 7662, 2015.
- [13] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0 incorporating errata set 1,” OpenID Foundation, Specification, 2014.
- [14] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” Internet Eng. Task Force, RFC 8446, 2018.
- [15] NIST, “Advanced encryption standard (AES),” Nat. Inst. Stand. Technol., FIPS Pub. 197, 2001.